

SSL/TLS & HTTPS

The Complete Security Infrastructure Masterclass

Beginner to Expert — DevOps | Cloud | Kubernetes | Security

Covers: Encryption Theory · Certificate Types · OpenSSL · Let's Encrypt · Kubernetes cert-manager

NGINX SSL · AWS ACM · mTLS · Service Mesh · Security Best Practices · Troubleshooting

Complete Practical Labs included

1. Introduction to SSL/TLS

Before you secure a single server, before you configure a single certificate, you need to understand what SSL/TLS actually is — not just what it does, but why it exists and how it works at a conceptual level. This section builds that foundation from scratch.

1.1 What is SSL?

SSL stands for Secure Sockets Layer. It was originally developed by Netscape Communications in 1994 to secure communications between web browsers and web servers. Before SSL, all HTTP traffic was sent in plain text — if you logged into a website over a public Wi-Fi network, anyone on that network could read your username, password, and every piece of data you sent.

SSL solved this by creating an encrypted channel between the client (browser) and server before any application data is sent. The data is scrambled using mathematical algorithms so that even if intercepted, it cannot be read without the correct decryption key.

1.2 SSL vs TLS — What is the Difference?

SSL and TLS are often used interchangeably, but they are technically different. TLS (Transport Layer Security) is the successor to SSL. Here is the complete version history:

Version	Year / Status
SSL 1.0	1994 — Never publicly released (had serious security flaws)
SSL 2.0	1995 — Released publicly, deprecated in 2011 (RFC 6176)
SSL 3.0	1996 — Deprecated in 2015 (RFC 7568) due to POODLE attack
TLS 1.0	1999 — Deprecated in 2020 (PCI DSS compliance requirement)
TLS 1.1	2006 — Deprecated in 2020
TLS 1.2	2008 — Still widely used, considered secure with proper configuration
TLS 1.3	2018 — Current recommended standard, significantly faster and more secure

□ Despite TLS being the modern standard since 1999, the industry still refers to certificates as 'SSL certificates'. This is purely historical terminology. When someone says 'install an SSL certificate', they mean TLS. When a product says 'SSL support', it means TLS.

1.3 Why HTTPS Exists

HTTP (HyperText Transfer Protocol) is the foundation of data communication on the web. HTTP by itself is completely insecure — everything is sent as plain readable text. HTTPS = HTTP + TLS. The S stands for Secure.

HTTPS provides three fundamental guarantees:

1. Confidentiality — Data is encrypted in transit. An attacker who intercepts the traffic sees only gibberish, not your password or credit card number.
2. Integrity — Data cannot be tampered with in transit. If an attacker modifies even one byte of the encrypted data, the recipient will detect the tampering and reject the message.
3. Authentication — The server proves its identity. When you connect to bank.com, HTTPS verifies you are actually talking to bank.com and not an impostor. This prevents man-in-the-middle attacks.

1.4 Encryption — The Core Concept

Encryption is the process of converting readable data (plaintext) into an unreadable format (ciphertext) using a mathematical algorithm and a key. Only someone with the correct key can reverse the process (decryption) and read the original data.

Symmetric Encryption

In symmetric encryption, the same key is used for both encryption and decryption. Think of a padlock where the same physical key locks and unlocks it.

Example: AES (Advanced Encryption Standard) — used in TLS for bulk data encryption after the handshake.

- Advantage: Very fast — suitable for encrypting large amounts of data
- Disadvantage: Key distribution problem — how do you securely share the key with the other party before you have a secure channel?

Asymmetric Encryption (Public Key Cryptography)

Asymmetric encryption uses a mathematically linked key pair: a public key and a private key. Data encrypted with the public key can ONLY be decrypted with the corresponding private key. Data signed with the private key can be verified with the public key.

- The public key can be shared with anyone — it is embedded in your SSL certificate
- The private key must NEVER leave your server — it is what proves you are the legitimate owner

Real-world analogy: A public key is like a padlock you hand out to everyone. Anyone can lock a message in the padlock and send it to you. But only you have the private key (the physical key) to open it.

How TLS Combines Both

TLS uses asymmetric encryption to solve the key distribution problem, then switches to symmetric encryption for performance:

4. During the TLS handshake, asymmetric cryptography (RSA or ECDHE) is used to securely exchange a session key
5. Once both parties have the session key, all subsequent data is encrypted with symmetric encryption (AES) which is 100-1000x faster
6. This hybrid approach gives you the security benefits of asymmetric crypto and the speed of symmetric crypto

1.5 Digital Signatures

A digital signature is a mathematical scheme that proves: (1) who created a piece of data and (2) that the data has not been modified since signing.

How it works in SSL certificates:

7. A Certificate Authority (CA) takes a hash of your certificate data
8. The CA encrypts that hash with its private key — this is the digital signature
9. Your browser decrypts the signature using the CA's public key (which is pre-installed in the browser)
10. If the decrypted hash matches a fresh hash of the certificate data, the signature is valid — the certificate is genuine and unmodified

1.6 The Chain of Trust

Browsers do not trust every CA in the world directly. Instead, they maintain a list of trusted Root Certificate Authorities (typically 50-100 entries). These Root CAs sign Intermediate CAs, and Intermediate CAs sign end-entity (server) certificates. This creates a chain:

Root CA (DigiCert) → Intermediate CA (DigiCert SHA2 Secure Server) → Your Website Certificate

Why use intermediates? If a Root CA private key were compromised, it would invalidate trust in every certificate it ever signed. By using intermediates, the Root CA private key can be kept in an air-gapped Hardware Security Module (HSM), signing only intermediate certificates rarely. If an intermediate is compromised, only its certificates need to be revoked.

1.7 The TLS Handshake — Overview

Before any encrypted data can be sent, the client and server must complete a TLS handshake to agree on encryption parameters and exchange keys. Here is the simplified flow:

11. Client Hello — Browser sends: TLS version supported, list of cipher suites supported, random number (client_random)
12. Server Hello — Server responds: chosen TLS version, chosen cipher suite, random number (server_random), and sends its certificate

13. Certificate Validation — Browser validates the server certificate: checks signature chain up to trusted Root CA, checks expiry date, checks hostname matches
14. Key Exchange — Browser generates a pre-master secret, encrypts it with server's public key, sends to server
15. Session Keys — Both sides independently compute the session keys from client_random + server_random + pre-master secret
16. Finished — Both sides confirm with a hash of the entire handshake — any tampering is detected
17. Encrypted Data — Application data (HTTP) now flows encrypted with the session keys

□ *In TLS 1.3, the handshake was redesigned to require only 1 round trip (1-RTT) instead of 2 (2-RTT in TLS 1.2), significantly reducing connection latency.*

2. Types of SSL/TLS Certificates

Not all SSL certificates are equal. They differ in what they validate, what domains they cover, and who issues them. Understanding these differences is essential for choosing the right certificate for your use case.

2.1 Validation Types

DV — Domain Validation

The most basic and fastest type. The CA only verifies that you control the domain name. No identity information about the organization is validated or included in the certificate.

- Issuance time: Minutes to a few hours
- Cost: Free (Let's Encrypt) to ~\$10/year
- Padlock: Shows padlock in browser but no organization name
- Use case: Blogs, personal sites, development, APIs, any site where basic encryption is sufficient
- Validation method: DNS record, HTTP file, or email to domain admin

OV — Organization Validation

The CA verifies both domain control AND the legal existence of the organization. The organization name is embedded in the certificate and visible to users who inspect it.

- Issuance time: 1-3 business days
- Cost: ~\$50-\$300/year
- Use case: Business websites, e-commerce where customer trust matters
- Validation: Domain control + business registration verification + phone call

EV — Extended Validation

The highest level of validation. The CA performs an extensive vetting process including legal, physical, and operational verification of the organization. Historically showed the company name in a green bar in the browser address bar (most modern browsers have removed the green bar display).

- Issuance time: 1-2 weeks
- Cost: ~\$200-\$1500/year
- Use case: Banks, payment processors, financial institutions, high-value targets
- Validation: Full business verification including jurisdiction, registration number, physical address, phone verification

□ As of 2019, Chrome and Firefox removed the distinctive EV indicator (green address bar with company name). EV certificates are still technically more validated but visually indistinguishable from OV in modern browsers.

2.2 Certificate Scope Types

Single Domain Certificate

Covers exactly one FQDN (Fully Qualified Domain Name). A certificate for `www.example.com` does NOT cover `example.com` — these are treated as different domains.

- Use case: Single website or service
- Example SANs: `www.example.com` only

Wildcard Certificate

Covers a domain and all direct subdomains at one level. A wildcard for `*.example.com` covers `api.example.com`, `www.example.com`, `mail.example.com` — but NOT `sub.api.example.com` (two levels deep).

- Use case: Organizations with many subdomains — `app.company.com`, `api.company.com`, `admin.company.com`
- Cost: Higher than single domain — typically \$100-\$500/year from commercial CAs
- Let's Encrypt: Supports wildcard via DNS-01 challenge only

☐ **A wildcard cert for `*.example.com` does NOT cover `example.com` itself. You need to include `example.com` as a SAN if needed.**

Multi-Domain / SAN Certificate

SAN stands for Subject Alternative Name. A SAN certificate can cover multiple completely different domains in a single certificate. Let's Encrypt issues SAN certificates by default, covering up to 100 domains per certificate.

- Example: `example.com`, `www.example.com`, `api.example.com`, `anotherdomain.org` — all in one cert
- Use case: Hosting providers, CDNs, organizations with multiple domain properties

2.3 Deployment Types

Self-Signed Certificates

A certificate signed by your own private key rather than a trusted CA. Your browser will show a security warning because no trusted third party has vouched for the certificate's authenticity.

- Cost: Free
- Issuance: Instant, no approval needed
- Use case: Internal services, development, Kubernetes pod-to-pod communication, testing
- Limitation: Browsers and clients will reject or warn unless explicitly trusted

Public CA Certificates

Issued by a publicly trusted Certificate Authority whose root certificate is pre-installed in browsers and operating systems. This is what you use for any public-facing website.

Internal Enterprise CA

Organizations can run their own internal CA (using tools like Microsoft Active Directory Certificate Services, HashiCorp Vault PKI, or cfssl). Internal CA certificates are trusted only by devices that have the internal root CA certificate installed.

- Use case: Internal microservices, mTLS, corporate intranets, VPN infrastructure
- Benefit: Complete control over certificate lifecycle, no per-certificate cost
- Requirement: Must distribute and maintain the root CA certificate on all trusting devices

3. Understanding SSL/TLS Components

When working with SSL certificates in practice, you encounter many different file types and concepts. This section explains every component in detail.

3.1 The Key Files Explained

File / Component	What It Is and What It Does
Private Key (.key)	The secret cryptographic key that NEVER leaves your server. Used to decrypt data that was encrypted with your public key. Losing control of this file means your certificate is compromised.
Certificate (.crt / .pem)	Your signed certificate file containing your public key plus identity information (domain name, organization, validity period). Sent to every client that connects to your server.
CSR (.csr)	Certificate Signing Request — contains your public key and identity info. You submit this to a CA who signs it and returns your certificate. The private key never leaves your server.
CA Bundle / Chain (.pem)	The intermediate certificates that connect your certificate to the Root CA. Required for browsers to build the full trust chain.
Full Chain (.pem)	Your certificate + intermediate certificates concatenated together. What you typically configure in nginx/Apache.
Root CA Certificate	The self-signed certificate of the top-level Certificate Authority. Pre-installed in browsers and OS trust stores.
Intermediate CA Certificate	Certificate issued by a Root CA, used to sign end-entity (server) certificates. Acts as a bridge in the trust chain.

3.2 File Formats

PEM Format

PEM (Privacy Enhanced Mail) is the most common format for SSL certificates on Linux/Unix systems. It is Base64-encoded DER data wrapped in header/footer lines.

```
-----BEGIN CERTIFICATE-----
MIIFazCCA1OgAwIBAgIRAIIQz7DSQONZRGPGu2OCiwAwDQYJKoZIhvcNAQELBQAw
TzELMAkGA1UEBhMCVVMxKTAnBgNVBAoTIEludGVybmV0IFN1Y3VyaXR5IFJlc2Vh
... (base64 encoded data) ...
-----END CERTIFICATE-----
```

PEM files can contain certificates, private keys, CSRs, or chains. The header tells you what type:

- -----BEGIN CERTIFICATE----- = certificate
- -----BEGIN RSA PRIVATE KEY----- = RSA private key (PKCS#1 format)

- -----BEGIN PRIVATE KEY----- = private key (PKCS#8 format)
- -----BEGIN CERTIFICATE REQUEST----- = CSR

DER Format

DER (Distinguished Encoding Rules) is the binary form of the certificate data. Same content as PEM but not Base64 encoded. Used primarily on Windows and Java applications. File extensions: .der, .cer.

PKCS#12 / PFX Format

PKCS#12 (file extension .p12 or .pfx) bundles the certificate, private key, and optionally the chain into a single password-protected file. Common on Windows and in Java keystores.

3.3 Format Conversion Commands

PEM to DER:

```
openssl x509 -in certificate.pem -outform der -out certificate.der
```

DER to PEM:

```
openssl x509 -in certificate.der -inform der -outform pem -out certificate.pem
```

PEM to PKCS#12 (bundle cert + key):

```
openssl pkcs12 -export -out certificate.pfx -inkey privatekey.pem \
-in certificate.pem -certfile ca-chain.pem
```

PKCS#12 to PEM (extract cert and key):

```
openssl pkcs12 -in certificate.pfx -nocerts -out privatekey.pem
openssl pkcs12 -in certificate.pfx -nokeys -out certificate.pem
```

View certificate contents:

```
openssl x509 -in certificate.pem -text -noout
```

3.4 Certificate Trust Store

A trust store is a collection of pre-trusted Root CA certificates maintained by an operating system or application. When a browser receives a server certificate, it traverses the chain until it finds a certificate in its trust store.

- Linux: /etc/ssl/certs/ — managed by ca-certificates package
- macOS: Keychain Access — System Roots
- Windows: Certificate Manager (certmgr.msc)
- Java: cacerts file in JRE installation
- Node.js: Uses OS trust store by default

Adding a custom CA to Ubuntu/Debian trust store:

```
sudo cp my-internal-ca.crt /usr/local/share/ca-certificates/  
sudo update-ca-certificates
```

Adding to RHEL/CentOS trust store:

```
sudo cp my-internal-ca.crt /etc/pki/ca-trust/source/anchors/  
sudo update-ca-trust
```

4. How SSL Certificates Are Created

Understanding the full lifecycle of a certificate — from key generation to issuance — is essential for anyone managing SSL infrastructure.

4.1 The Complete Certificate Lifecycle

18. Generate a private key on your server
19. Create a CSR (Certificate Signing Request) from the private key
20. Submit the CSR to a Certificate Authority
21. Complete the validation process (DNS, HTTP, or email)
22. CA signs your CSR with its Intermediate CA private key and returns your certificate
23. Configure your server with the private key, certificate, and CA chain
24. Certificate expires — renew before expiry

4.2 Generating Keys and CSRs with OpenSSL

Step 1: Generate an RSA Private Key

```
# Generate 4096-bit RSA private key (recommended for production)
openssl genrsa -out private.key 4096

# Generate with password protection
openssl genrsa -aes256 -out private.key 4096
```

Step 2: Generate an ECDSA Private Key (Modern Alternative)

ECDSA (Elliptic Curve Digital Signature Algorithm) keys provide equivalent security to RSA with much smaller key sizes, resulting in faster TLS handshakes.

```
# Generate EC key using P-256 curve (recommended)
openssl ecparam -name prime256v1 -genkey -noout -out ec-private.key

# Or P-384 for higher security
openssl ecparam -name secp384r1 -genkey -noout -out ec-private.key
```

Step 3: Generate a CSR

```
openssl req -new -key private.key -out certificate.csr \
  -subj "/C=IN/ST=Maharashtra/L=Mumbai/O=MyCompany/CN=example.com"
```

CSR field meanings:

- C = Country code (2 letters: IN, US, GB)
- ST = State or Province
- L = Locality / City

- O = Organization name (must match legal business name for OV/EV)
- OU = Organizational Unit (optional)
- CN = Common Name — the primary domain name

Step 4: Generate Self-Signed Certificate

```
# Self-signed cert valid for 365 days
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout private.key -out certificate.crt \
  -subj "/C=IN/ST=MH/L=Mumbai/O=TestCo/CN=localhost"
```

Step 5: Create Your Own CA and Sign Certificates

This is how you build an internal PKI — useful for development environments and microservices.

```
# 1. Generate Root CA private key
openssl genrsa -aes256 -out ca.key 4096

# 2. Generate Root CA self-signed certificate (10 year validity)
openssl req -x509 -new -nodes -key ca.key -sha256 -days 3650 \
  -out ca.crt -subj "/C=IN/O=MyInternalCA/CN=My Root CA"

# 3. Generate server private key
openssl genrsa -out server.key 2048

# 4. Generate server CSR
openssl req -new -key server.key -out server.csr \
  -subj "/C=IN/O=MyCompany/CN=api.internal.company.com"

# 5. Sign server CSR with Root CA
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key \
  -CAcreateserial -out server.crt -days 365 -sha256
```

Viewing and Verifying Certificates

```
# View certificate details
openssl x509 -in certificate.crt -text -noout

# View CSR details
openssl req -in certificate.csr -text -noout

# Verify certificate matches private key
openssl x509 -noout -modulus -in certificate.crt | openssl md5
openssl rsa -noout -modulus -in private.key | openssl md5
# These two hashes must match

# Test SSL connection
openssl s_client -connect example.com:443 -servername example.com
```

5. Certificate Authorities (CA)

A Certificate Authority is an organization that issues digital certificates, verifying the identity of the certificate owner. Understanding how CAs work is essential for managing trust in your infrastructure.

5.1 The CA Ecosystem

CA / Provider	Details
Let's Encrypt	Free, automated, open CA backed by ISRG. Issues DV certificates only. 90-day validity. Best for automated renewal. Used by millions of websites.
DigiCert	Enterprise CA, premium pricing. Offers DV/OV/EV. Known for excellent customer support and reliability. Used by large enterprises and financial institutions.
Sectigo (formerly Comodo)	One of the largest CAs by market share. Offers all certificate types at competitive prices. Popular for commercial deployments.
GlobalSign	Enterprise CA with strong presence in Europe. Excellent for OV and EV certificates.
Cloudflare	Provides free SSL to sites using Cloudflare's CDN/proxy. Also offers Origin Certificates for backend-to-Cloudflare connections.
AWS ACM	Free certificate manager for AWS services (ALB, CloudFront, API Gateway). Cannot be downloaded/exported — only for AWS resources.
GoDaddy	Popular registrar offering SSL at various price points. Known for ease of use for small businesses.
ZeroSSL	Alternative to Let's Encrypt. Free DV certificates with ACME protocol support. Offers a REST API as alternative to ACME.

5.2 Validation Methods

DNS Validation (DNS-01)

You prove domain ownership by creating a specific DNS TXT record. The CA queries DNS for this record to confirm you control the domain. This is the **ONLY** method that works for wildcard certificates.

```
# Let's Encrypt DNS challenge example
# You must create a TXT record:
acme-challenge.example.com. IN TXT "abc123randomtoken"
```

HTTP Validation (HTTP-01)

You prove domain ownership by serving a specific file at a well-known URL. The CA makes an HTTP request to confirm the file is there.

```
# You must serve a file at:  
http://example.com/.well-known/acme-challenge/TOKEN_VALUE  
# File content: TOKEN VALUE.KEY AUTHORIZATION
```

Email Validation

For OV/EV certificates. The CA sends a verification email to administrative addresses for the domain (admin@, administrator@, webmaster@, postmaster@, hostmaster@) or the WHOIS contact email.

5.3 Certificate Revocation

Certificates can be revoked before their expiry date if the private key is compromised or the certificate was issued incorrectly. There are two mechanisms to check revocation status:

CRL — Certificate Revocation List

A periodically updated list of revoked certificate serial numbers published by the CA. Browsers can download this list and check if a certificate has been revoked. Disadvantage: CRLs can be large and are updated infrequently (sometimes daily), creating a window where a revoked cert might still be trusted.

OCSP — Online Certificate Status Protocol

A real-time protocol that allows clients to query the CA about the revocation status of a specific certificate. Faster and more efficient than CRL for individual lookups.

OCSP Stapling

Instead of the browser querying the CA (which reveals browsing history and adds latency), the web server periodically fetches its own OCSP response and 'staples' it to the TLS handshake. The browser gets the revocation proof without contacting the CA directly.

```
# Enable OCSP stapling in nginx  
ssl_stapling on;  
ssl_stapling_verify on;  
resolver 8.8.8.8 8.8.4.4 valid=300s;
```

6. Let's Encrypt and Certificate Automation

Let's Encrypt revolutionized web security by making SSL certificates free and automatable. Understanding Let's Encrypt and the ACME protocol is essential for any modern DevOps engineer.

6.1 What is Let's Encrypt?

Let's Encrypt is a free, automated, and open Certificate Authority run by the Internet Security Research Group (ISRG). It was launched in 2016 with the mission to encrypt the entire web. As of 2024, it has issued over 3 billion certificates.

- Certificates are free — no payment required
- Certificates are valid for 90 days (shorter than commercial CAs to encourage automation)
- Issuance and renewal can be fully automated via the ACME protocol
- Only issues DV certificates (no OV or EV)
- Wildcard certificates supported via DNS-01 challenge only

6.2 The ACME Protocol

ACME (Automated Certificate Management Environment) is the protocol Let's Encrypt uses. It standardizes the process of proving domain ownership and receiving certificates. Certbot is the most popular ACME client.

6.3 Installing Certbot on Ubuntu

```
# Install snapd if not present
sudo apt update
sudo apt install snapd

# Install certbot via snap (recommended)
sudo snap install --classic certbot
sudo ln -s /snap/bin/certbot /usr/bin/certbot
```

6.4 Obtaining Certificates

With nginx (Automatic nginx configuration)

```
sudo certbot --nginx -d example.com -d www.example.com
```

With Apache

```
sudo certbot --apache -d example.com -d www.example.com
```


Standalone mode (no web server running)

```
sudo certbot certonly --standalone -d example.com
```

Webroot mode (web server must serve /.well-known/acme-challenge/)

```
sudo certbot certonly --webroot -w /var/www/html -d example.com
```

Wildcard certificate via DNS challenge

```
sudo certbot certonly --manual --preferred-challenges dns \
-d '*.example.com' -d example.com
```

6.5 Certificate Files Location

Let's Encrypt stores certificates in `/etc/letsencrypt/live/domain/`:

File	Contents
cert.pem	Your domain certificate only
privkey.pem	Your private key — keep secure
chain.pem	Intermediate certificates only
fullchain.pem	cert.pem + chain.pem concatenated — use this in nginx/Apache

6.6 Auto-Renewal

Let's Encrypt certificates expire after 90 days. Certbot installs a systemd timer that runs renewal checks twice daily.

```
# Check renewal timer status
sudo systemctl status snap.certbot.renew.timer

# Test renewal without actually renewing
sudo certbot renew --dry-run

# Force renewal
sudo certbot renew --force-renewal

# List all certificates and their expiry
sudo certbot certificates
```

Manual cron-based renewal (alternative)

```
# Add to crontab -e
0 12 * * * /usr/bin/certbot renew --quiet && systemctl reload nginx
```

7. TLS Handshake — Deep Dive

The TLS handshake is the most technically complex part of SSL/TLS. Understanding it deeply lets you debug connection issues, optimize performance, and configure cipher suites correctly.

7.1 TLS 1.2 Handshake (Full Detail)

TLS 1.2 requires 2 round trips (2-RTT) before application data can flow:

Round Trip 1:

25. CLIENT → SERVER: ClientHello — TLS version, random 32-byte nonce (client_random), session ID, list of cipher suites in preference order, list of compression methods
26. SERVER → CLIENT: ServerHello — chosen TLS version, chosen cipher suite, random 32-byte nonce (server_random), session ID
27. SERVER → CLIENT: Certificate — the server's certificate chain (leaf cert + intermediate certs)
28. SERVER → CLIENT: ServerKeyExchange — (if needed) additional key exchange parameters (for DHE/ECDHE)
29. SERVER → CLIENT: ServerHelloDone — signals end of server's hello phase

Round Trip 2:

30. CLIENT → SERVER: ClientKeyExchange — pre-master secret encrypted with server's public key (for RSA) OR client's key share (for DHE/ECDHE)
31. CLIENT → SERVER: ChangeCipherSpec — signals switch to encrypted communication
32. CLIENT → SERVER: Finished — HMAC hash of entire handshake, encrypted
33. SERVER → CLIENT: ChangeCipherSpec — same from server side
34. SERVER → CLIENT: Finished — server's HMAC hash of handshake, encrypted
35. APPLICATION DATA — HTTP (or whatever) now flows encrypted

7.2 TLS 1.3 Handshake (Improved)

TLS 1.3 achieves 1-RTT (one round trip) by removing unnecessary legacy features and merging steps:

36. CLIENT → SERVER: ClientHello — includes key share immediately (no separate step)
37. SERVER → CLIENT: ServerHello + EncryptedExtensions + Certificate + CertificateVerify + Finished — all in one flight
38. CLIENT → SERVER: Finished — client confirms
39. APPLICATION DATA flows

TLS 1.3 also supports 0-RTT (Zero Round Trip Time Resumption) for reconnecting clients using a pre-shared session ticket — application data can be sent with the very first message. Caveat: 0-

RTT data is vulnerable to replay attacks and should only be used for idempotent requests (GET, not POST).

7.3 Cipher Suites Explained

A cipher suite is a combination of algorithms used in a TLS connection. In TLS 1.2 format: TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384

Component	Meaning
ECDHE	Key exchange algorithm: Elliptic Curve Diffie-Hellman Ephemeral — provides Perfect Forward Secrecy
RSA	Authentication algorithm: how the server's identity is verified
AES_256_GCM	Bulk encryption: AES 256-bit in Galois/Counter Mode (authenticated encryption)
SHA384	MAC (Message Authentication Code) algorithm for data integrity

Recommended Cipher Suites (Modern)

```
# TLS 1.3 (ciphers are fixed — not configurable)
TLS_AES_256_GCM_SHA384
TLS_CHACHA20_POLY1305_SHA256
TLS_AES_128_GCM_SHA256

# TLS 1.2 recommended ciphers
TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384
TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256
TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256
```

7.4 Perfect Forward Secrecy (PFS)

PFS ensures that even if your server's private key is compromised in the future, an attacker cannot decrypt past recorded traffic. Without PFS (using RSA key exchange), an attacker who captures encrypted traffic today and later steals your private key can decrypt all of it retroactively.

With PFS (using ECDHE key exchange), a new ephemeral key pair is generated for every session. The session key is never transmitted and cannot be derived from the long-term private key. Destroying the ephemeral key destroys all ability to decrypt that session.

❑ **Ensure your server only uses ECDHE or DHE cipher suites, not bare RSA key exchange. Verify with: `openssl s_client -connect yourdomain.com:443` and look for 'Protocol: TLSv1.3' or ECDHE in the cipher.**

8. SSL/TLS in Kubernetes

Kubernetes has its own ecosystem of tools and patterns for managing SSL certificates. From basic TLS Secrets to fully automated cert-manager with Let's Encrypt, this section covers everything.

8.1 TLS Secrets

The foundational way to store a certificate and private key in Kubernetes is a Secret of type `kubernetes.io/tls`.

Creating a TLS Secret from existing files

```
kubectl create secret tls my-tls-secret \  
  --cert=path/to/certificate.crt \  
  --key=path/to/private.key \  
  --namespace=my-namespace
```

TLS Secret YAML manifest

```
apiVersion: v1  
kind: Secret  
metadata:  
  name: my-tls-secret  
  namespace: my-namespace  
type: kubernetes.io/tls  
data:  
  tls.crt: <base64-encoded-certificate>  
  tls.key: <base64-encoded-private-key>
```

To base64 encode your certificate for the YAML:

```
cat certificate.crt | base64 -w 0  
cat private.key | base64 -w 0
```

8.2 Ingress with TLS

Once you have a TLS Secret, you reference it in an Ingress resource to enable HTTPS for your service.

Basic Ingress with TLS

```
apiVersion: networking.k8s.io/v1  
kind: Ingress  
metadata:  
  name: my-app-ingress  
  namespace: my-namespace  
  annotations:  
    nginx.ingress.kubernetes.io/ssl-redirect: "true"  
    nginx.ingress.kubernetes.io/force-ssl-redirect: "true"  
spec:
```

```

ingressClassName: nginx
tls:
- hosts:
  - app.example.com
  secretName: my-tls-secret
rules:
- host: app.example.com
  http:
    paths:
    - path: /
      pathType: Prefix
      backend:
        service:
          name: my-app-service
          port:
            number: 80

```

8.3 Installing NGINX Ingress Controller

```

# Using Helm
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update

helm upgrade --install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx --create-namespace \
  --set controller.service.type=LoadBalancer

```

8.4 cert-manager — Certificate Automation

cert-manager is a Kubernetes controller that automates the management and issuance of TLS certificates. It integrates with Let's Encrypt, Vault, Venafi, and self-signed issuers. cert-manager watches for Certificate resources and automatically handles validation, issuance, and renewal.

Installing cert-manager

```

# Install via Helm (recommended)
helm repo add jetstack https://charts.jetstack.io
helm repo update

helm upgrade --install cert-manager jetstack/cert-manager \
  --namespace cert-manager --create-namespace \
  --set installCRDs=true \
  --version v1.14.0

```

```

# Verify installation
kubectl get pods -n cert-manager
# Should show: cert-manager, cert-manager-cainjector, cert-manager-webhook - all
Running

```

ClusterIssuer — Let's Encrypt Production

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: admin@example.com
    privateKeySecretRef:
      name: letsencrypt-prod-account-key
    solvers:
      - http01:
          ingress:
            class: nginx
```

ClusterIssuer — Let's Encrypt Staging (for testing)

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-staging
spec:
  acme:
    server: https://acme-staging-v02.api.letsencrypt.org/directory
    email: admin@example.com
    privateKeySecretRef:
      name: letsencrypt-staging-account-key
    solvers:
      - http01:
          ingress:
            class: nginx
```

☐ Always test with the staging issuer first. Let's Encrypt production has rate limits (50 certs per domain per week). The staging issuer has much higher limits but issues untrusted certificates.

Ingress with cert-manager annotation (automatic)

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-app-ingress
  namespace: production
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
```

```

ingressClassName: nginx
tls:
- hosts:
  - app.example.com
  secretName: app-tls-cert
rules:
- host: app.example.com
  http:
    paths:
    - path: /
      pathType: Prefix
      backend:
        service:
          name: my-app
          port:
            number: 80

```

When this Ingress is applied, cert-manager automatically: creates an ACME challenge, creates a temporary pod to serve the HTTP-01 challenge, waits for Let's Encrypt to validate, stores the issued certificate in the 'app-tls-cert' Secret, and configures auto-renewal 30 days before expiry.

Explicit Certificate resource

```

apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: app-cert
  namespace: production
spec:
  secretName: app-tls-cert
  issuerRef:
    name: letsencrypt-prod
    kind: ClusterIssuer
  commonName: app.example.com
  dnsNames:
  - app.example.com
  - www.app.example.com
  duration: 2160h # 90 days
  renewBefore: 720h # Renew 30 days before expiry

```

Wildcard Certificate with DNS-01 Challenge

```

apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-dns
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: admin@example.com
    privateKeySecretRef:

```

```
    name: letsencrypt-dns-key
  solvers:
  - dns01:
      route53:
        region: us-east-1
        hostedZoneID: ZXXXXXXXXXXXXXXXXX
```

cert-manager Troubleshooting

```
# Check certificate status
kubectl get certificate -n production
kubectl describe certificate app-cert -n production

# Check CertificateRequest
kubectl get certificaterequest -n production

# Check ACME challenge status
kubectl get challenge -n production
kubectl describe challenge -n production

# Check cert-manager logs
kubectl logs -n cert-manager deploy/cert-manager
```


9. SSL/TLS on EC2 / Traditional Servers

9.1 SSL on nginx

Basic nginx SSL configuration

```
server {
    listen 80;
    server_name example.com www.example.com;
    return 301 https://$host$request_uri; # HTTP to HTTPS redirect
}

server {
    listen 443 ssl http2;
    server_name example.com www.example.com;

    # Certificate files
    ssl_certificate      /etc/letsencrypt/live/example.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/example.com/privkey.pem;

    # Modern TLS configuration
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:
                ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:
                ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305;
    ssl_prefer_server_ciphers off;
    ssl_session_timeout 1d;
    ssl_session_cache shared:SSL:50m;

    # OCSP Stapling
    ssl_stapling on;
    ssl_stapling_verify on;
    ssl_trusted_certificate /etc/letsencrypt/live/example.com/chain.pem;
    resolver 8.8.8.8 8.8.4.4 valid=300s;

    # HSTS (1 year)
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains"
always;
    add_header X-Frame-Options DENY;
    add_header X-Content-Type-Options nosniff;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Verify nginx config and reload

```
sudo nginx -t  
sudo systemctl reload nginx
```

9.2 SSL on Apache

```
<VirtualHost *:80>  
    ServerName example.com  
    Redirect permanent / https://example.com/  
</VirtualHost>  
  
<VirtualHost *:443>  
    ServerName example.com  
  
    SSLEngine on  
    SSLCertificateFile      /etc/letsencrypt/live/example.com/cert.pem  
    SSLCertificateKeyFile   /etc/letsencrypt/live/example.com/privkey.pem  
    SSLCertificateChainFile /etc/letsencrypt/live/example.com/chain.pem  
  
    SSLProtocol             all -SSLv3 -TLSv1 -TLSv1.1  
    SSLCipherSuite          ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256  
    SSLHonorCipherOrder     off  
    SSLSessionTickets       off  
  
    Header always set Strict-Transport-Security "max-age=63072000"  
</VirtualHost>
```

9.3 Let's Encrypt on EC2 (Full Walkthrough)

40. Launch EC2 instance with Ubuntu 22.04, open ports 22, 80, 443 in Security Group
41. Point your domain's A record to the EC2 public IP
42. Install nginx: `sudo apt install -y nginx`
43. Install certbot: `sudo snap install --classic certbot`
44. Run: `sudo certbot --nginx -d example.com -d www.example.com`
45. Certbot automatically edits nginx config and sets up auto-renewal
46. Test: `curl -I https://example.com`

10. Applying SSL to Applications

10.1 SSL Termination Patterns

Pattern	Description
SSL Termination at Load Balancer	TLS is decrypted at the LB/ingress. Backend services communicate over plain HTTP. Simplest to implement. Use when backend network is trusted (private VPC).
SSL Passthrough	TLS traffic passes through the LB to the backend. Backend terminates TLS. Load balancer cannot inspect or modify content. Required for mTLS to backend.
End-to-End Encryption	TLS from client to LB, AND TLS from LB to backend. Maximum security. Required for compliance in many regulated industries.
SSL Re-encryption	LB terminates client TLS, then re-encrypts to backend with different cert. Common with internal CA for backend certs.

10.2 Node.js HTTPS Server

```
const https = require('https');
const fs = require('fs');
const express = require('express');

const app = express();

app.get('/', (req, res) => res.send('Secure Hello!'));

const options = {
  key: fs.readFileSync('/etc/ssl/private.key'),
  cert: fs.readFileSync('/etc/ssl/certificate.crt'),
  ca: fs.readFileSync('/etc/ssl/ca-bundle.crt') // if needed
};

https.createServer(options, app).listen(443, () => {
  console.log('HTTPS server running on port 443');
});
```

10.3 Python Flask with SSL

```
from flask import Flask
import ssl

app = Flask(__name__)
```

```
@app.route('/')
def hello():
    return 'Secure Flask!'

if __name__ == '__main__':
    context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    context.load_cert_chain('/etc/ssl/certificate.crt', '/etc/ssl/private.key')
    app.run(host='0.0.0.0', port=443, ssl_context=context)
```

10.4 Mounting TLS Secrets in Kubernetes Pods

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-secure-app
spec:
  template:
    spec:
      containers:
      - name: app
        image: my-app:latest
        env:
        - name: TLS_CERT_PATH
          value: /etc/ssl/tls.crt
        - name: TLS_KEY_PATH
          value: /etc/ssl/tls.key
        volumeMounts:
        - name: tls-certs
          mountPath: /etc/ssl
          readOnly: true
      volumes:
      - name: tls-certs
        secret:
          secretName: my-tls-secret
```

11. Cloud and Load Balancer SSL

11.1 AWS Certificate Manager (ACM)

ACM provides free SSL/TLS certificates for use with AWS services. Important: ACM certificates cannot be exported or used on EC2 directly — they are only for AWS-managed services.

- Supported services: ALB, NLB, CloudFront, API Gateway, Elastic Beanstalk
- Auto-renewal: ACM automatically renews certificates before expiry
- Certificate validation: DNS validation (recommended, automatic with Route 53) or email

Request ACM certificate via AWS CLI

```
aws acm request-certificate \  
  --domain-name example.com \  
  --subject-alternative-names www.example.com api.example.com \  
  --validation-method DNS \  
  --region us-east-1
```

11.2 SSL on AWS ALB

The Application Load Balancer terminates TLS, then forwards HTTP traffic to targets. Configure via the Listeners settings:

47. Add HTTPS listener on port 443
48. Select ACM certificate
49. Configure security policy (ELBSecurityPolicy-TLS13-1-2-2021-06 recommended)
50. Add HTTP listener (port 80) with redirect to HTTPS rule

11.3 Cloudflare SSL

Cloudflare acts as a TLS-terminating reverse proxy. Traffic from users to Cloudflare is encrypted. Traffic from Cloudflare to your origin can be configured:

SSL Mode	Description
Off	No encryption anywhere — never use this
Flexible	User to Cloudflare: HTTPS. Cloudflare to origin: HTTP. Quick but origin is not encrypted.
Full	Both encrypted, but Cloudflare does not validate your origin cert. Self-signed accepted.
Full (Strict)	Both encrypted, and Cloudflare validates your origin cert. Use Cloudflare Origin Certificate or publicly trusted cert.

SSL Mode	Description
Strict (HTTPS Only)	Origin must have a valid, non-expired, publicly trusted certificate.

Cloudflare Origin Certificate

A free certificate issued by Cloudflare, valid for 15 years, trusted ONLY by Cloudflare. Install on your origin server for Full (Strict) mode when using Cloudflare proxy.

12. Security Best Practices

12.1 TLS Version Configuration

❑ **Disable SSLv3, TLSv1.0, and TLSv1.1. These versions have known vulnerabilities (POODLE, BEAST, CRIME) and are deprecated by PCI DSS since 2020.**

```
# nginx: enforce TLS 1.2 and 1.3 only
ssl_protocols TLSv1.2 TLSv1.3;
```

12.2 HSTS — HTTP Strict Transport Security

HSTS is a header that tells browsers to ONLY use HTTPS for a domain for a specified period. Prevents SSL stripping attacks where an attacker downgrades HTTPS to HTTP.

```
# Add to nginx server block (after HTTPS is confirmed working)
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload"
always;
```

HSTS preload: submitting your domain to browsers' built-in HSTS list means users get HTTPS enforcement even on first visit. Submit at: <https://hstspreload.org>

❑ **Set HSTS ONLY after you are sure HTTPS works permanently. If you ever revert to HTTP, browsers will refuse to connect for up to 2 years (max-age period).**

12.3 Private Key Security

- Private keys must have 600 permissions: `chmod 600 /etc/ssl/private.key`
- Private keys must be owned by root or the web server user only
- Never commit private keys to Git — ever
- Store private keys in secrets management (HashiCorp Vault, AWS Secrets Manager, Kubernetes Secrets)
- Use Hardware Security Modules (HSMs) for production CA keys
- Rotate certificates and keys annually at minimum

12.4 Common SSL Vulnerabilities

Vulnerability	Description and Mitigation
POODLE (2014)	Exploits SSL 3.0 padding oracle. Mitigation: Disable SSLv3 completely.
HEARTBLEED (2014)	OpenSSL memory leak — attackers could read server memory including private keys. Mitigation: Update OpenSSL.

Vulnerability	Description and Mitigation
BEAST (2011)	TLS 1.0 CBC mode attack. Mitigation: Disable TLS 1.0, use RC4 (now also weak) or AES-GCM.
CRIME/BREACH	Compression oracle attacks. Mitigation: Disable TLS compression (ssl_comp_disable).
DROWN (2016)	Uses SSLv2 to attack TLS. Mitigation: Disable SSLv2/3 globally.
LOGJAM (2015)	Weak DH key exchange (512-bit). Mitigation: Use 2048-bit+ DH params, prefer ECDHE.
Expired Certificate	Causes browser errors, breaks apps. Mitigation: Automate renewal with certbot/cert-manager.
Certificate Mismatch	Certificate CN/SAN doesn't match hostname. Mitigation: Include all necessary SANs.
Mixed Content	HTTPS page loading HTTP resources. Mitigation: Upgrade all assets to HTTPS.

13. Troubleshooting SSL/TLS

13.1 Diagnostic Commands

OpenSSL — The Primary SSL Debug Tool

```
# Test basic TLS connection
openssl s_client -connect example.com:443 -servername example.com

# Show full certificate chain
openssl s_client -connect example.com:443 -showcerts

# Test specific TLS version
openssl s_client -connect example.com:443 -tls1_2
openssl s_client -connect example.com:443 -tls1_3

# Check certificate expiry
echo | openssl s_client -connect example.com:443 2>/dev/null | \
  openssl x509 -noout -dates

# Check what cipher was negotiated
openssl s_client -connect example.com:443 2>/dev/null | grep 'Cipher is'

# View local certificate details
openssl x509 -in certificate.crt -text -noout
```

curl for HTTPS Testing

```
# Basic HTTPS test
curl -I https://example.com

# Verbose output (shows handshake details)
curl -v https://example.com

# Skip certificate verification (for self-signed - development only)
curl -k https://localhost:443

# Use specific CA cert
curl --cacert /path/to/ca.crt https://internal.service

# Test with specific TLS version
curl --tlsv1.3 https://example.com
```

13.2 Common Errors and Solutions

Error	Cause and Solution
ERR_CERT_AUTHORITY_INVALID	Browser doesn't trust the CA. Self-signed cert or missing intermediate chain. Solution: Use a trusted CA or install the root cert in the browser/OS trust store.
ERR_CERT_DATE_INVALID	Certificate is expired or not yet valid. Solution: Renew certificate. Check server time is correct with: date command.
ERR_CERT_COMMON_NAME_INVALID	Certificate hostname doesn't match the domain. Solution: Ensure SAN includes all domain names. CN alone is not sufficient in modern browsers.
SSL_ERROR_RX_RECORD_TOO_LONG	Server is not TLS — likely plain HTTP on port 443. Solution: Check server is configured for SSL, not sending HTTP on SSL port.
HANDSHAKE_FAILURE	No common cipher suites between client and server. Solution: Review and update cipher suite configuration on server.
Certificate chain incomplete	Intermediate certificates not served. Solution: Use fullchain.pem in nginx, not just cert.pem. Verify: openssl s_client -showcerts.
Mixed Content Warning	HTTPS page loading HTTP resources. Solution: Update all resource URLs to HTTPS or use protocol-relative URLs.
net::ERR_CERT_REVOKED	Certificate was revoked by CA. Solution: Issue new certificate with a new key pair.

13.3 Kubernetes-Specific Troubleshooting

```
# Check cert-manager certificate status
kubectl get certificate --all-namespaces
kubectl describe certificate my-cert -n production

# Check if TLS secret exists and has correct keys
kubectl get secret my-tls-secret -n production -o yaml

# Decode and inspect certificate in secret
kubectl get secret my-tls-secret -n production \
  -o jsonpath='{.data.tls\.crt}' | base64 -d | openssl x509 -text -noout

# Check ingress TLS configuration
kubectl describe ingress my-ingress -n production
```

```
# Check ACME challenges
kubectl get challenges --all-namespaces
kubectl describe challenge -n production

# cert-manager controller logs
kubectl logs -n cert-manager -l app=cert-manager --tail=100
```

14. Advanced SSL/TLS Concepts

14.1 mTLS — Mutual TLS

Standard TLS verifies the server's identity to the client. Mutual TLS (mTLS) adds the reverse — the server also verifies the client's identity. Both parties present certificates and verify each other. This is the foundation of zero-trust networking.

mTLS use cases:

- Service-to-service authentication in microservices
- API authentication without bearer tokens
- VPN and corporate network access
- Kubernetes pod-to-pod communication (via service mesh)

nginx mTLS configuration

```
server {
    listen 443 ssl;
    ssl_certificate      /etc/ssl/server.crt;
    ssl_certificate_key  /etc/ssl/server.key;

    # mTLS: require client certificate
    ssl_client_certificate /etc/ssl/client-ca.crt;
    ssl_verify_client on;

    location / {
        # Access client cert info in headers
        proxy_set_header X-SSL-Client-CN  $ssl_client_s_dn_cn;
        proxy_set_header X-SSL-Client-Verify $ssl_client_verify;
        proxy_pass http://backend;
    }
}
```

14.2 Service Mesh TLS (Istio)

A service mesh like Istio automatically injects an Envoy sidecar proxy into every pod. These sidecars handle all TLS transparently — your application code communicates over plain HTTP internally, while Istio provides mTLS between all pods automatically.

```
# Enable mTLS in Istio namespace
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: production
spec:
  mtls:
```

```
mode: STRICT
```

With mode: STRICT, Istio requires mTLS for ALL traffic in the namespace. Plain HTTP between services is rejected. This is the gold standard for microservice security.

14.3 SPIFFE/SPIRE

SPIFFE (Secure Production Identity Framework For Everyone) is an open standard for service identity in dynamic infrastructure. SPIRE is the reference implementation. Instead of managing certificates manually, SPIRE assigns a cryptographic identity (SVID - SPIFFE Verifiable Identity Document) to each workload based on its attestation.

- Each workload gets a unique SPIFFE ID: `spiffe://example.org/ns/production/sa/api-service`
- SVIDs are short-lived and automatically rotated
- Works across clouds, on-premises, and in containers

14.4 Internal PKI with HashiCorp Vault

HashiCorp Vault's PKI secrets engine can act as an internal CA, issuing short-lived certificates programmatically — perfect for microservices.

```
# Enable PKI secrets engine
vault secrets enable pki

# Configure max TTL
vault secrets tune -max-lease-ttl=87600h pki

# Generate root CA
vault write pki/root/generate/internal \
  common_name="My Internal CA" ttl=87600h

# Create role for certificate issuance
vault write pki/roles/my-service \
  allowed_domains=internal.company.com \
  allow_subdomains=true max_ttl=72h

# Issue certificate
vault write pki/issue/my-service \
  common_name=api.internal.company.com
```

15. Complete Practical Labs

Lab 1: Self-Signed Certificate for Local Development

Goal: Set up HTTPS on localhost with a self-signed certificate trusted by your browser.

51. Generate CA and server certificate
52. Trust the CA in your OS/browser
53. Configure nginx with the certificate

```
# Step 1: Generate local CA
openssl genrsa -out local-ca.key 2048
openssl req -x509 -new -nodes -key local-ca.key -sha256 -days 1825 \
    -out local-ca.crt -subj "/CN=Local Development CA"

# Step 2: Create cert config with SAN for localhost
cat > localhost.conf <<EOF
[req]
distinguished_name = req_distinguished_name
req_extensions = v3_req
prompt = no
[req_distinguished_name]
CN = localhost
[v3_req]
subjectAltName = @alt_names
[alt_names]
DNS.1 = localhost
DNS.2 = *.localhost
IP.1 = 127.0.0.1
EOF

# Step 3: Generate server key and CSR
openssl genrsa -out localhost.key 2048
openssl req -new -key localhost.key -out localhost.csr -config localhost.conf

# Step 4: Sign with local CA
openssl x509 -req -in localhost.csr -CA local-ca.crt -CAkey local-ca.key \
    -CAcreateserial -out localhost.crt -days 365 -sha256 -extensions v3_req \
    -extfile localhost.conf

# Step 5: Trust CA in Ubuntu
sudo cp local-ca.crt /usr/local/share/ca-certificates/
sudo update-ca-certificates
```

Lab 2: Let's Encrypt on EC2 with nginx

```
# Prerequisites: EC2 Ubuntu 22.04, ports 80/443 open, domain pointing to EC2 IP
```

```

# Install nginx
sudo apt update && sudo apt install -y nginx

# Create basic nginx config
sudo tee /etc/nginx/sites-available/mysite <<EOF
server {
    listen 80;
    server_name example.com www.example.com;
    root /var/www/html;
    index index.html;
}
EOF
sudo ln -s /etc/nginx/sites-available/mysite /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx

# Install certbot
sudo snap install --classic certbot
sudo ln -s /snap/bin/certbot /usr/bin/certbot

# Obtain certificate
sudo certbot --nginx -d example.com -d www.example.com

# Verify auto-renewal
sudo certbot renew --dry-run

```

Lab 3: Full Kubernetes SSL with cert-manager

```

# Step 1: Install NGINX Ingress Controller
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm upgrade --install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx --create-namespace

# Step 2: Install cert-manager
helm repo add jetstack https://charts.jetstack.io
helm upgrade --install cert-manager jetstack/cert-manager \
  --namespace cert-manager --create-namespace --set installCRDs=true

# Step 3: Apply ClusterIssuer
kubectl apply -f - <<EOF
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: admin@example.com
    privateKeySecretRef:
      name: letsencrypt-prod-key

```

```
    solvers:
      - http01:
          ingress:
            class: nginx
EOF

# Step 4: Deploy app with TLS Ingress
kubectl apply -f - <<EOF
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-app
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: nginx
  tls:
    - hosts: [app.example.com]
      secretName: app-tls
  rules:
    - host: app.example.com
      http:
        paths:
          - path: / pathType: Prefix
            backend:
              service: {name: my-app, port: {number: 80}}
EOF

# Step 5: Watch certificate issuance
kubectl get certificate -w
```


16. Final Summary, Checklists & Interview Guide

16.1 SSL/TLS Production Checklist

- ✓ Use TLS 1.2 and TLS 1.3 only — disable all older versions
- ✓ Use ECDHE cipher suites for Perfect Forward Secrecy
- ✓ Serve the full certificate chain (cert + intermediates)
- ✓ Enable HSTS with long max-age (63072000 minimum)
- ✓ Enable OCSP Stapling
- ✓ Private key permissions: 600, owned by service user
- ✓ Automate certificate renewal (certbot/cert-manager)
- ✓ Monitor certificate expiry (alert at 30 days remaining)
- ✓ Use 2048-bit RSA minimum, prefer 4096-bit or ECDSA P-256
- ✓ Never store private keys in version control
- ✓ Test with SSL Labs (ssllabs.com/ssltest) and aim for A+ rating
- ✓ Enable HTTP to HTTPS redirect (301 permanent)

16.2 Top Interview Questions

Q: Explain the TLS handshake in simple terms.

A: TLS handshake is like a secure introduction ceremony before two people share secrets. The client says 'I can speak these languages (cipher suites)'. The server picks one and shows its ID card (certificate). The client verifies the ID with a trusted authority (CA). They then secretly exchange a key using asymmetric crypto. From then on, they use that shared key to encrypt all conversation using fast symmetric encryption.

Q: What is Perfect Forward Secrecy and why does it matter?

A: PFS means that if your server's private key is stolen in the future, it cannot be used to decrypt past recorded traffic. Achieved by using ephemeral ECDHE key exchange — a new temporary key pair is generated per session, destroyed after use, and never stored. Without PFS (bare RSA key exchange), a future key compromise decrypts all past sessions. Every modern server should use ECDHE.

Q: A pod in Kubernetes shows CrashLoopBackOff related to SSL. How do you debug?

A: Check the TLS Secret exists with: `kubectl get secret -n namespace`. Verify it has both `tls.crt` and `tls.key` keys. Decode and inspect the cert: `kubectl get secret name -o jsonpath='{.data.tls.crt}' |`

base64 -d | openssl x509 -noout -dates. Check if cert is expired or hostname doesn't match. Check pod logs for specific SSL error. Verify the file path the app expects matches the Secret mount path.

Q: What is mTLS and when would you use it?

A: Mutual TLS means both client and server present certificates and verify each other's identity. Use it for: service-to-service authentication in microservices (both pods prove identity), API authentication without rotating bearer tokens, zero-trust network architecture where nothing is trusted by network position alone. In Kubernetes, Istio automates mTLS between all pods transparently.

Q: What is cert-manager and how does auto-renewal work?

A: cert-manager is a Kubernetes controller that watches Certificate resources and manages the full lifecycle. It integrates with issuers (Let's Encrypt, Vault, self-signed). When a certificate expires within renewBefore time (e.g., 30 days), cert-manager triggers a new ACME challenge, gets a fresh certificate from the issuer, and updates the Kubernetes Secret — all without human intervention. Ingress controllers pick up the new cert automatically.

Q: Explain the difference between SSL termination, passthrough, and end-to-end TLS.

A: Termination: TLS decrypted at the load balancer, plain HTTP to backend — simplest, fast, suitable when backend network is trusted. Passthrough: TLS passes through LB to backend — LB cannot inspect content, required for mTLS. End-to-end: TLS from client to LB, re-encrypted LB to backend — maximum security, required for some compliance frameworks. Choose based on your security requirements and backend trust model.

16.3 Beginner Learning Roadmap

- 54. Week 1: Understand HTTP vs HTTPS, encryption basics (symmetric, asymmetric), what a certificate contains
- 55. Week 2: Generate self-signed certificates with OpenSSL, learn PEM/DER formats, inspect certificates
- 56. Week 3: Get a free Let's Encrypt certificate for a domain, configure nginx HTTPS, understand renewal
- 57. Week 4: Set up cert-manager on Kubernetes, create ClusterIssuers, apply TLS to Ingress
- 58. Week 5: Deep dive TLS handshake, cipher suites, Perfect Forward Secrecy, HSTS
- 59. Week 6: Learn mTLS concepts, Istio service mesh basics, internal PKI with Vault
- 60. Week 7: Security hardening, SSL Labs testing, vulnerability study, production best practices
- 61. Week 8: Architecture review — design SSL strategy for a full microservices platform

Security is not a product. It is a process. Automate, monitor, and never stop learning.